Algorithms

Lecture 1: Growth of Functions and Big-O

Demo Lecturer

Semester 2, 2026

Housekeeping

- Weekly quiz: released after the Friday lecture, due the following Friday
- Tutorial: Wednesday 15:00 – bring your attempts at the tute questions
- Discussion board and consultation hours: see the course page
- Textbook: any standard algorithms textbook (readings are listed per lecture)

Why Do We Care About Efficiency?

Two ways to compute $1 + 2 + \cdots + n$:

Loop	**Formula**
add $1, 2, \ldots, n$ one by one	compute $n(n+1)/2$
about n additions	one multiplication, one division

Both are correct. For $n = 10^9$ only one of them is practical.

Question: how do we compare algorithms *without* running them?

Input Size and Basic Operation

We measure cost as a function of the **input size** n, counting the **basic operation** – the operation executed most often in the innermost loop.

Problem	Input size	Basic operation
Search a list	number of items n	key comparison
Multiply two matrices	matrix dimension n	scalar multiplication
Test if n is prime	number of bits of n	division

Worst, Best and Average Case

```
function find(A, x, n)
    j <- 0
    while j < n
        if A[j] = x
            return j
        j <- j + 1
    return -1
```

- Basic operation: the comparison `A[j] = x`
- **Worst case**: x is absent, so n comparisons
- **Best case**: x is in the first position, so 1 comparison

Big-O: The Definition

$O(g(n))$ is the *set* of functions that grow no faster than g.

Definition

$t(n) \in O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $t(n) \leq c \cdot g(n)$ for all $n > n_0$.

Example. $3n + 10 \in O(n)$.

For $n > 10$ we have $3n + 10 < 3n + n = 4n$, so choose $c = 4$ and $n_0 = 10$.

Big-Omega and Big-Theta

- $t(n) \in \Omega(g(n))$: $t(n) \geq c \cdot g(n)$ for all $n > n_0$ – a **lower** bound
- $t(n) \in \Theta(g(n))$: both $O(g)$ and $\Omega(g)$ – the **exact** order of growth

Caution. Writing $3n \in O(n^2)$ is true but loose. Use Θ when you can show both bounds.

Common Growth Rates and Summary

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

- Count the basic operation as a function of input size
- Ignore constant factors and small inputs
- Next lecture: binary search – an algorithm with $\Theta(\log n)$ worst case